

SD Programming 101

A tutorial with worked example programs - lessons 1 to 6, from hello world to reading a file with a select list.

This page is a tutorial. The rest of the `User` set is reference: each page covers one subject and every statement on it. Here we put them together into programs that do something, and each program builds on the one before.

Every program on this page compiles and runs on SD Core for Windows W1.0-0. The output shown is what SD printed. If you type the program in, compile it and run it, you will see the same thing.

The reference pages are linked at the end of each section. Read them when you want the full story on a statement or function — this page shows what you need to make the programs work and no more.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part.

Before you start

You need an SD session and a `bp` file. If you have signed in and created a file called `customers` as the Introduction suggests, you already have `bp` — every account gets one.

```
sd
```

You land at the `:` prompt. That is TCL — the command processor. From here you compile, catalogue and run programs. The programs themselves are written in SDBasic, which is a different language from TCL.

Your `bp` file is an ordinary Windows folder. You can write programs in `ed` from inside SD, in `edit` or `micro` if your account can run external programs, or in any text editor you like — Notepad, VS Code, anything. The folder is on disk at:

```
C:\ProgramData\SD\user_accounts\<account>\bp
```

1. Hello world

The smallest program:

```
program hello
  print 'hello, world'
```

```
end
```

Type it into `ed` — `ed bp hello`, `i`, type the three lines, a full stop on its own line to stop inserting, `fi` to file and exit. Or save it as `hello` in the `bp` folder with a text editor.

Compile and run it:

```
basic bp hello
run bp hello
```

```
hello, world
```

Every program starts with a declaration — `program` here — and ends with `end`. Between them are statements. `print` sends a line to the terminal. The string is in single quotes; double quotes work too, but single quotes are what the rest of the documentation uses and what SD's own source uses.

`program` is optional. A source record with no declaration is a program. But every program on this page has one, because naming the program is how you know what it is when it is catalogued and called from another program.

2. Variables and input

A program that asks your name and says hello back:

```
program hello2
  print 'what is your name?' :
  input name
  print 'hello, ' : name
end
```

```
basic bp hello2
run bp hello2
```

```
what is your name? dave
hello, dave
```

`input` reads a line from the terminal into the variable `name`. The colon at the end of the first `print` stops it printing a newline, so the question mark is on the same line as the answer.

Variables are not declared. A variable that has never been assigned is an **unassigned variable** — reading it is an error, not an empty string. This is deliberate: it catches typos.

The colon `:` in a `print` statement is the concatenation operator for output. It does not add spaces — it joins the pieces with nothing between them. To put a space between *hello*, and the name, the string itself carries the space:

```
'hello, '.
```

3. Variables and arithmetic

SDBasic variables are **type variant** — a variable holds whatever type the last assignment gave it, and the type can change from one line to the next. A variable that held a string can be used in arithmetic on the next line: the value is converted to numeric form for the calculation without changing the variable itself.

Numeric values are held as **integers wherever possible**. Conversion to floating point occurs when a result is non-integer or too large to store as an integer. This is invisible — you do not declare a variable as integer or real.

```
program maths
  print 'first number?' :
  input a
  print 'second number?' :
  input b
  print 'sum      = ' : a + b
  print 'product  = ' : a * b
  print 'quotient = ' : a / b
  print 'remainder= ' : mod(a, b)
end
```

```
run bp maths
```

```
first number? 17
second number? 5
sum      = 22
product  = 85
quotient = 3.4
remainder= 2
```

`mod()` is a function, not a statement — it is called inside an expression and returns a value. SDBasic has a full set of math functions: `abs`, `int`, `sqrt`, `sin`, `cos`, `tan`, `ln`, `exp`, `pwr`, and more. The full list is in SD Basic - Math Functions.

The redundant-looking type-forcing idiom. Because conversion happens on use, not on assignment, a string of digits used in a tight loop converts on every iteration. SDBasic programs often contain apparently redundant statements that force the conversion once:

```
a = a + 0 ;* Convert to numeric form s = s : "" ;* Convert to string form
```

The first makes `a` numeric so the next arithmetic use does not convert again; the second makes `s` a string. Neither changes the value — they change how it is held.

4. A program that reads a file

This is where SDBasic starts to earn its keep. The `customers` file from the Introduction holds records. Each record is a dynamic array — a string with field marks in it. Reading a record gives you the whole string; extracting a field gives you one piece of it.

```
program show.customer
  open 'customers' to f.cust else
    print 'cannot open customers'
    stop
  end

  print 'customer id?' :
  input id

  read rec from f.cust, id then
    print 'name : ' : rec<1>
    print 'phone : ' : rec<2>
    print 'city : ' : rec<3>
  end else
    print 'no such customer'
  end
end
```

```
run bp show.customer
```

```
customer id? 1001
name  : Acme Supplies
phone : 555-1234
city  : Springfield
```

WHAT JUST HAPPENED

`open` looks up `customers` in the VOC and gives you a file variable. The `else` branch fires if the file does not exist — and **it is compulsory**. A program that omits the `else` does not compile.

`read` fetches the whole record by id. The `then` branch fires if the record exists; the `else` branch fires if it does not.

`rec<1>` is **field extraction** — the angle brackets are the shorthand for `extract(rec, 1, 0, 0)`. Field 1, field 2, field 3. Dynamic arrays are covered in SD Basic - Dynamic Arrays.

5. A program that writes

```
program add.customer
  open 'customers' to f.cust else
    print 'cannot open customers'
    stop
  end

  print 'new customer id?' :
  input id

  print 'name?' :
  input name
  print 'phone?' :
  input phone
  print 'city?' :
  input city

  rec = name : @fm : phone : @fm : city
  write rec on f.cust, id
  print 'saved'
end
```

```
run bp add.customer
```

```
new customer id? 1002
name? Widget Corp
phone? 555-5678
city? Shelbyville
saved
```

`@fm` is the **field mark constant** — character 254. Building a record is joining fields with it. `write` stores the whole record; the old record, if any, is replaced.

THE MARKS

Three mark characters divide a dynamic array:

	NAME	CONSTANT	CODE
field mark	<code>@fm</code>	254	
value mark	<code>@vm</code>	253	
subvalue mark	<code>@sm</code>	252	

A field contains values; a value contains subvalues. `rec<1>` gets the first field; `rec<1,2>` gets the second value in the first field; `rec<1,2,1>` gets the first subvalue in that value.

6. A program with a select list

Reading one record by id is useful. Reading every record in a file is how you produce reports. A select list is the mechanism:

```
program list.customers
  open 'customers' to f.cust else
    print 'cannot open customers'
    stop
  end

  select f.cust to 1
  print 'id' : ' ' : 'name' : ' ' : 'city'
  print '---' : ' ' : '----' : ' ' : '----'

  loop
    readnext id from 1 else exit
    read rec from f.cust, id then
```

```
        print id : ' ' : rec<1> : ' ' : rec<3>
    end
repeat

    clearselect 1
end
```

```
run bp list.customers
```

```
id name city
--- ----
1001 Acme Supplies Springfield
1002 Widget Corp Shelbyville
```

`select` builds a list of every record id in the file. `readnext` reads one id at a time from the list.

`readnext` **is a statement, not a function**, and its `else` branch is what ends the loop — it fires when the list is exhausted. `loop` / `readnext ... else exit` / `repeat` is the standard pattern for walking a select list. Writing it as `readnext(id)` does not fail helpfully: the compiler reads an unknown function call as a matrix reference and complains that `READNEXT` is not referenced in a `dim` statement.

`clearselect 1` clears list 1 when the program is done. A select list is **session state**: it survives the program that made it. Clearing it is good manners.

Continued in

SD Programming 101 - Routines and an Application — lessons 7 to 10 — subroutines, functions, error handling and a small application.